

COXWAVE AI 리서치·제품화 엔지니어 직무과제

멀티에이전트 시스템의 오류 위치 및 유형 탐지

- 실행 기록을 기반으로 오류 위치 및 유형을 탐지하고 수치화하는 파이프라인 구축
- 네 오류 유형을 모두 라벨한 평가 데이터셋 구축

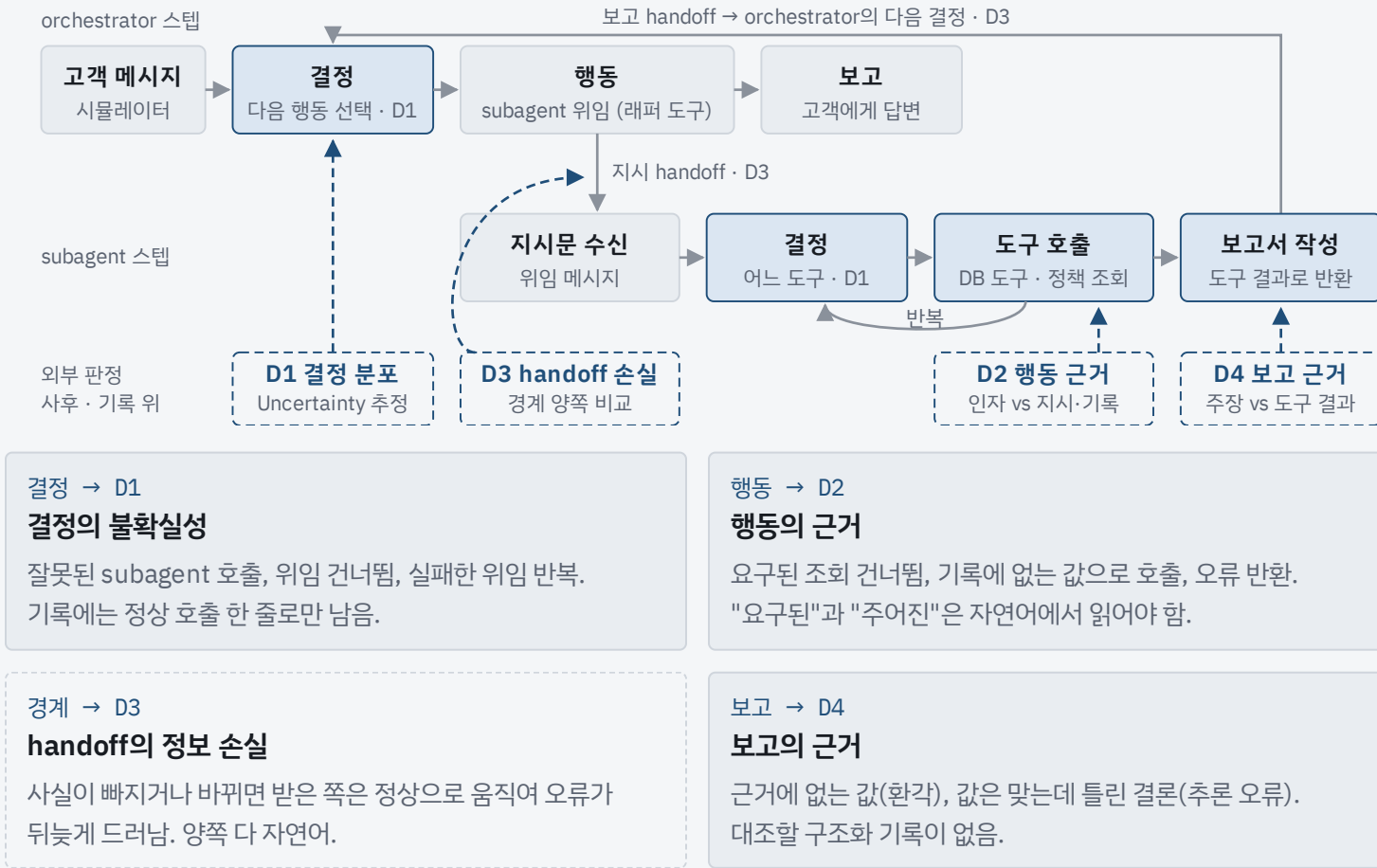
1 배경

실행 기록만으로 오류의 위치와 유형을 찾을 수 있는가

1.1 연구 세부 주제 | 배경

에이전트가 동작하는 과정에서는 다양한 이유로 응답에 오류가 생깁니다. 어디서 시작됐고 어떤 유형인지를 탐지하고 수치화해야 하는데, 운영에서 남는 것은 실행 기록뿐이고 정답도 사람 라벨도 없습니다. 대상은 orchestrator가 subagent에게 일을 나누어 맡기는 순차 멀티에이전트 시스템이며, 탐지는 실행이 끝난 뒤 저장된 기록 위에서 수행합니다.

1.2 문제 정의 | Deep Agents 실행 흐름에서 오류가 생기는 네 자리와 외부 판정자가 붙는 곳



1.3 차별점 (Novelty)

- 결정의 불확실성을 실행 모델 자신의 확률로 잴**
공개 트레이스 벤치마크는 출력만 남아 다른 모델로 재생해야 확률을 얻음.
실행과 채점에 같은 가중치·채팅 템플릿·도구 스키마를 쓰고, 후보가 유한하므로 샘플링 없이 결정 지점마다 짧은 채점으로 끝남
- 판정 주체를 탐지 대상에 맞춤**
결정·행동은 코드가 판정. 구조화된 도구 호출에는 문자열 대조가 LLM보다 정밀했고 모든 플래그에 근거 포인터가 붙음. handoff 경계는 자연어대 자연어라 LLM이 판정하며, 직접 비교가 사실 추출 후 집합 비교보다 정확. 어느 탐지 대상에 무엇이 맞는지를 데이터로 정함
- 정답과 라벨 없이 동작**
어느 검사도 정답·보상·라벨을 입력으로 받지 않고 임계값은 자기 분포의 백분위.
라벨은 이 실험의 평가에만 사용. 성공과 실패가 섞인 운영 로그를 사후 검토하는 제품에 그대로 얹힘
- 성공 실행을 포함한 완전 관측 데이터셋과 생성 파이프라인**
모델이 실제로 본 입력, 도구 호출·결과 원문, 공유 파일 스냅샷까지 담고 결정·행동·경계·보고 네 탐지 대상을 모두 라벨. 성공 실행이 있어 오탐율과 회복 분석이 가능. 모델 라벨이라 사람 라벨의 대체가 아니라 출발점

관련 연구

계열	대표 연구	요지	본 연구와의 관계
실패 귀속 벤치마크	Who&When [1] TraceElephant [2]	실패한 멀티에이전트 트레이스에서 책임 에이전트와 결정적 스텝을 맞추는 과제. TraceElephant는 입력까지 담은 완전 관측 트레이스로 귀속 정확도를 최대 76 % 높임	두 벤치마크는 실패 트레이스만 담아 오탐을 개념이 없고, 확률은 다른 모델로 근사함. 이 제안은 직접 실행으로 두 한계를 피하며, "완전 관측이 필요하다"는 결론은 공유함
트레이스 오류 분류	MAST [3] TRAIL [4]	트레이스 단위 오류 분류체계. MAST는 1,600여 트레이스에서 14개 실패 모드 (이력 손실, 추론-행동 불일치, 검증 실패 등)를 정리하였고, TRAIL은 841개 오류를 추론(환각 등), 실행, 계획·조정으로 나누어 스텝 단위로 표시함	본 연구의 라벨 범주 세 가지(handoff, tool, reasoning_stability)는 TRAIL의 계획·조정, 실행, 추론 구분과 하나씩 대응함. 두 분류체계를 그대로 쓰지는 않고 스텝 단위 라벨 규약(§4.1.3)을 따로 정함
불확실성 기반 탐지	semantic entropy [5] SelfCheckGPT [6]	반복 생성의 불일치로 환각을 탐지함	샘플링에 기반함. 이 제안은 유한 행동 집합에서만 불확실성을 쓰고, 텍스트 환각에는 쓰지 않음(과신 환각에 취약하기 때문)

References

[1] Zhang, S. et al. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. ICML 2025. arXiv:2505.00212

[2] Chen, M. et al. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. ACL 2026. arXiv:2604.22708

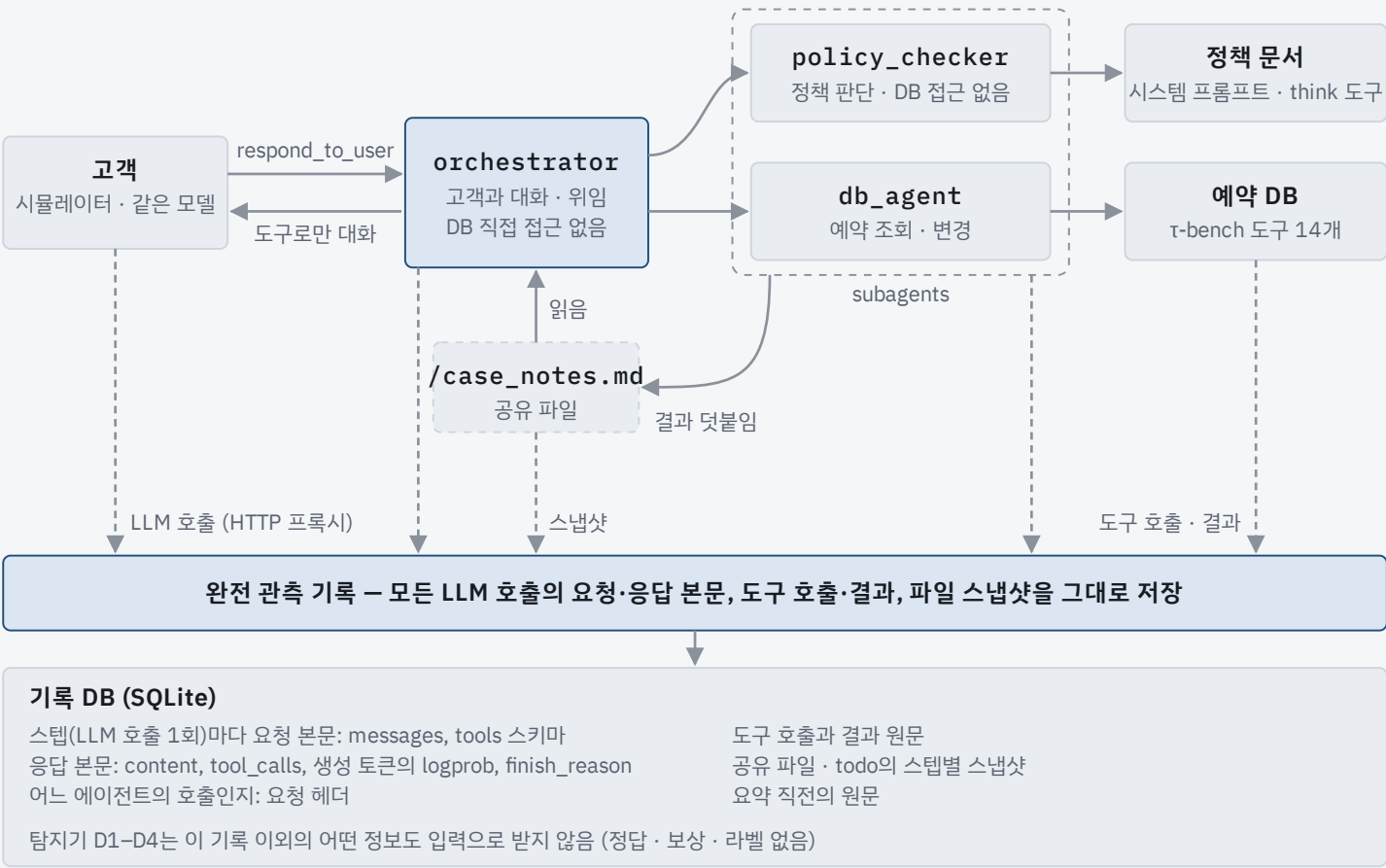
[3] Cemri, M. et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025

[4] Deshpande, D. et al. TRAIL: Trace Reasoning and Agentic Issue Localization. arXiv:2505.08638, 2025

[5] Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature 630, 625–630 (2024)

[6] Manakul, P. et al. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. EMNLP 2023. arXiv:2303.08896

τ-bench airline을 orchestrator→subagent 그래프로 실행하고 모든 호출을 기록합니다



3.1 실행 환경

3.1.1 왜 이 벤치마크인가

정책 판단과 데이터베이스 조회가 섞인 고객 응대 과제입니다. 에이전트는 문서로 주어진 항공사 정책을 지키면서 시뮬레이션 고객과 대화하고, 대화가 끝난 뒤 DB의 최종 상태가 정답과 같아야 성공이라 성패가 객관적입니다.

3.1.2 subagent

orchestrator는 policy_checker, db_agent라는 이름의 도구를 부르고, 그 도구가 subagent 그래프를 실행한 뒤 보고서를 도구 결과로 돌려줍니다. 내장 task는 대상 subagent를 인자 안에 숨겨 "누구를 부르는가"의 확률을 잴 수 없기 때문입니다.

3.1.3 Backbone LLM

실행 모델과 고객 시뮬레이터 모두 Qwen3-32B입니다 (thinking 끄, temperature 0). 시뮬레이터는 τ-bench의 사용자 시뮬레이터를 같은 모델로 다시 구현한 것입니다.

3.2 완전 관측 기록

탐지기가 읽는 것은 왼쪽 기록 DB뿐입니다. logprob은 모델이 각 토큰에 매긴 로그 확률이며, 실행 모델과 같은 가중치로 다시 채점할 수 있도록 모델이 본 입력 전체를 보존합니다.

3 제안

오류 탐지기 종류 및 구성요소

3.3 탐지기 구성

탐지기	탐지 대상	대조 상대	판정 주체	묻는 질문	점수
D1 결정 분포	결정	모델 자신의 확률 분포	코드	다음 행동을 고를 때 얼마나 흔들렸는가	결정이 흔들린 정도 (1 - 실제 선택의 확률 등)
D2 행동 근거	행동(도구 호출)	기록(지시문, 주어진 값)	코드	이 도구 호출은 지시와 기록이 뒷받침하는가	어긋남 있음/없음 (0/1) + 근거
D3 handoff 정보 손실	handoff 경계	경계 반대편의 텍스트	LLM	지시와 보고가 넘어가는 동안 어떤 사실이 빠지거나 바뀌었는가	사실 손실 비율 (1 - fidelity) + 누락·변형 목록
D4 보고 근거	보고	근거(도구 결과, 지시문)	LLM	보고의 주장 하나하나가 도구 결과와 지시에서 나온 것인가	근거 없는 주장의 비율

탐지기별 계산 방법

D1 결정 분포
후보별 확률 재채점

후보는 부를 수 있는 도구 이름과 "도구 없이 답하기". 문맥을 고정하고 후보마다 실행 모델의 확률을 다시 계산. 샘플링 없음. 실제 선택의 확률이 낮거나 분포가 퍼져 있으면 흔들린 결정.

D2 행동 근거
세 가지 어긋남 검사

(1) 지시가 요구한 도구를 부르지 않음 (2) 기록 어디에도 없는 값(예약번호, 날짜 등)으로 호출 (3) 호출이 오류로 돌아옴. 하나라도 해당하면 1. orchestrator의 위임은 식별자 없는 위임과 실패 뒤 재위임을 검사.

D3 handoff 정보 손실
경계 양쪽 텍스트 비교

지시→전제, 보고→다음 행동의 두 경계. 판정 LLM이 앞 텍스트의 사실을 나열하고 뒤 텍스트에서 보존·누락·변형을 표시. fidelity = 보존된 사실의 비율.

D4 보고 근거
주장 단위 근거 판정

판정 LLM이 보고를 주장 단위로 나누고 각 주장을 근거 있음 / 근거에서 유도됨 / 근거 없음 / 근거와 모순으로 분류. 값은 맞는데 결론이 틀린 경우를 겨누는 유일한 탐지기.

라벨 없는 운영 설계와 평가 데이터셋

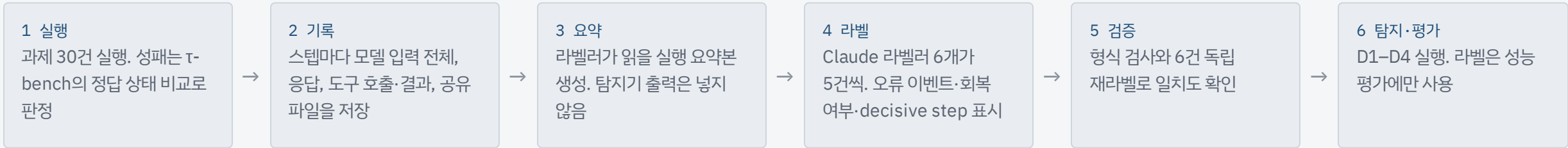
3.4 라벨 없는 운영 | 설계 원칙

- 탐지기는 정답이나 라벨 없이 실행 기록만 읽음. 경고 기준(임계값)은 각 탐지기 점수 분포의 상위 5 % 또는 10 %로 정함
- 경고마다 근거가 된 스텝과 값을 함께 표시. 점수 순으로 사람이 소량만 검수하고 기준을 조정하는 운영 루프
- 전제는 §3.2의 완전 관측 기록. 모델 출력만 남는 로그로는 D1·D2·D3을 돌릴 수 없음

3.5 평가 데이터셋 | 공개 벤치마크와 다른 점

- 성공한 실행도 포함하므로 잘못 올린 경고(오탐)의 비율과 오류에서 회복한 경우를 잴 수 있음
- 모델이 본 입력과 도구 결과 원문을 그대로 담아 같은 모델로 다시 채점 가능
- 네 탐지 대상 모두에 라벨이 있어, 아직 잘 잡지 못하는 보고 지점의 후속 탐지기도 같은 기준으로 평가 가능
- 라벨은 사람이 아니라 Claude가 달았음. 사람 라벨의 대체가 아니라 출발점

3.5 평가 데이터셋 | 생성 파이프라인과 규모



구축된 데이터셋 규모 (τ-bench airline)

30 회 실행 기록 (성공 6 / 실패 24)	826 개 라벨 대상 스텝	215 건 오류 이벤트 (handoff 77 · 도구 82 · 추론 56)	24 회 decisive step이 있는 실패 실행	60 건 handoff 정보 손실 정답 사례 (D3 평가용)
---------------------------------	-------------------	---	---------------------------------	---

4 결과 및 평가

탐지기 성능 평가를 위한 oracle 데이터 구축

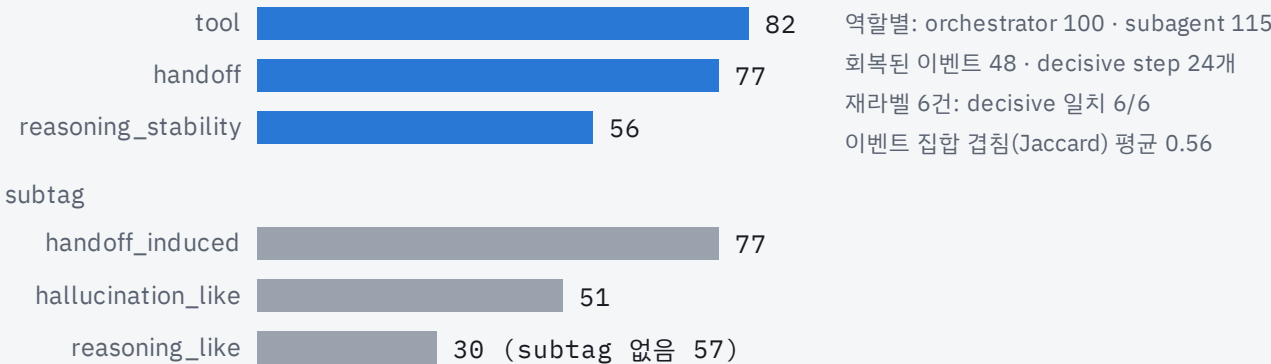
4.1 실험 설계

4.1.1 실험 규모

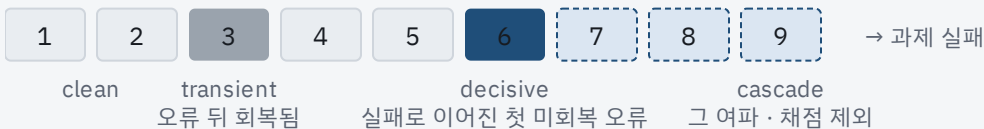
τ-bench airline 30회(배치 1·2 각 15건). 성공 6, 실패 24. 스텝 1,029개, 실행당 중앙값 31.5스텝.

4.1.2 라벨링 프로토콜 — 라벨 분포 (airline 215건)

범주 (215건)



4.1.3 decisive step과 스텝 클래스



decisive step은 실패한 실행에서 회복되지 못한 첫 오류 이벤트이며, 탐지기가 반드시 잡아야 하는 스텝입니다. 그 뒤의 스텝은 여파(cascade)로 보아 채점에서 뺍니다. 회복된 오류는 transient, 성공 실행의 미회복 오류는 error로 둡니다.

4.1.3 라벨링 프로토콜 — 절차

Claude 라벨러 6개가 실행 5건씩. 실행 요약본만 보고 탐지기 출력은 보지 않음(블라인드)

오류 이벤트마다 회복 여부, 회복된 스텝, 기록에서 어긋난 부분의 인용을 기록

독립 재라벨 6건에서 decisive step은 모두 일치. 이벤트 집합 겹침 0.56은 두 번째 라벨러가 같은 오류의 반복을 빠뜨린 것이라, 반복도 스텝마다 라벨하도록 규약을 고정

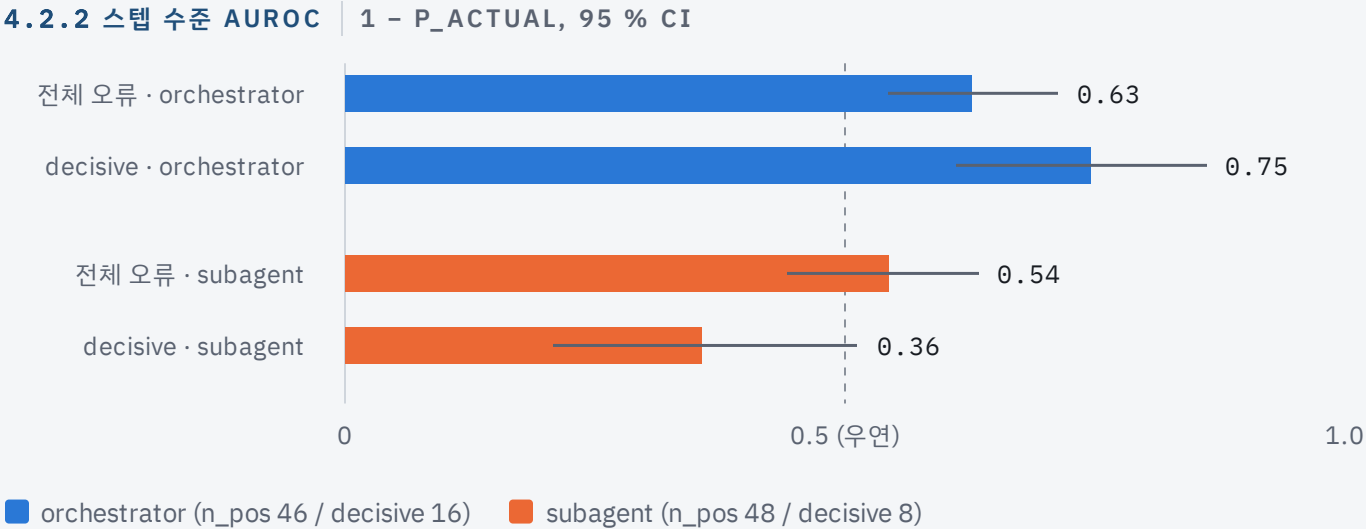
4.1.4 지표 정의

스텝 AUROC는 탐지기 점수로 오류 스텝(전체 오류 또는 decisive만)을 깨끗한 스텝과 얼마나 잘 가르는지이며, 95 % 신뢰구간은 부트스트랩 1,000회로 구합니다. 0/1 플래그를 내는 D2는 정밀도·재현율·F1·오탐율로 잡니다.

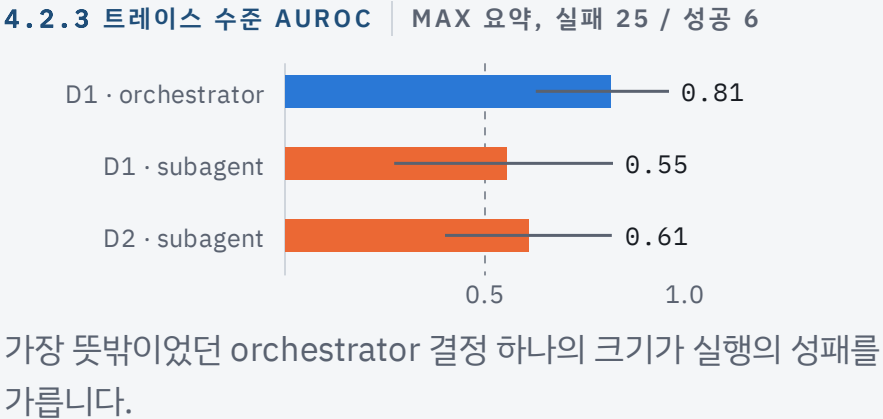
4.1.5 D1 teacher forcing 검증

결정 지점 906개 모두에 teacher forcing으로 후보 확률을 다시 계산했고, 그 입력이 실행 시 입력과 974/974 일치합니다. 같은 요청을 다시 teacher forcing하면 값이 약 0.005 흔들리므로 D1 값은 소수 둘째 자리까지만 유효합니다.

orchestrator의 결정이 흔들린 정도는 실패를 예고하고, subagent에서는 신호가 없습니다



orchestrator는 매 스텝 여러 후보 사이에서 정하는 자리라 실패가 결정의 흔들림으로 나타납니다. 세 점수 가운데 1 - p_actual이 일관되게 가장 높는데, 결정적 실패의 상당수가 분포는 뽕족한데 그 뽕족한 쪽이 아닌 행동을 실행한 경우이기 때문입니다.

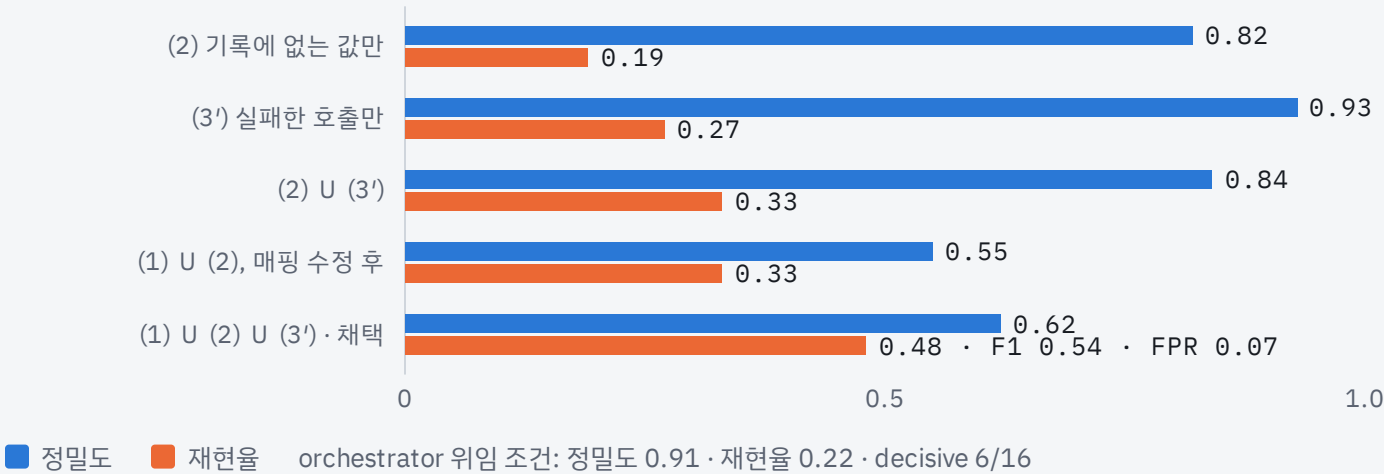


가장 뜻밖이었던 orchestrator 결정 하나의 크기가 실행의 성패를 가릅니다.

subagent는 지시문으로 다음 행동이 거의 정해져 분포가 뽕족합니다. 오류는 어느 도구를 부르는가가 아니라 어떤 인자로 부르고 어떤 결론을 내는가에서 나므로 D1에 보이지 않습니다. 탐지 지연은 중앙값 4스텝으로, D1은 결정적 실패 그 자리보다 직후의 흔들림을 잡습니다.

subagent의 실수는 확신에 찬 잘못된 호출이라 코드 대조가 잡습니다

4.2.8 D2 구성 요소 선택 | 정밀도와 재현율 (SUBAGENT, 양성 48 / 음성 190)



4.2.9 SUBTAG 서술 분석 | 무엇을 잡는가

- 환각성 subagent 오류 29건 중 23건이 (2)에 걸림. 예: 고객이 성만 말했는데 `get_user_details(user_id='rossi_123')`
- 지어낸 값은 발언이 아니라 **인자**에 있었음. 폐기한 발언 값 검사는 같은 49건 중 38건을 "근거 있음"으로 봄
- (3')는 대개 (2)의 증상. 57건 중 32건이 겹침

무엇을 놓치는가

- 인자의 출처는 맞는데 선택이 틀린 경우(다른 예약의 날짜) 10건
- 보고 지점과 경계 지점의 오류 15건은 D2의 질문 밖
- 잡음의 주된 원인은 (1)의 지시문→도구 군 정규식. 남은 오탐 14건 대부분

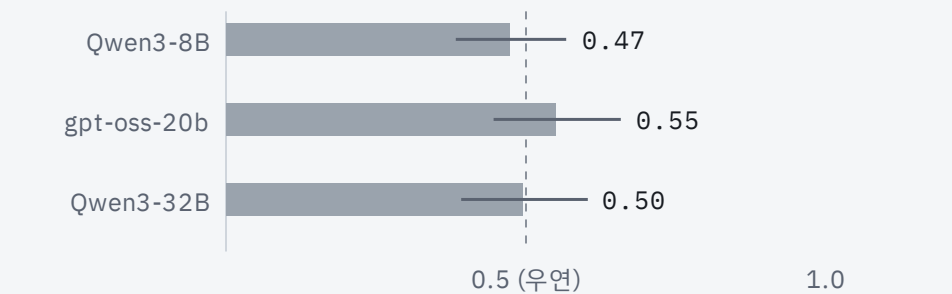
D3은 손실을 재고, D4는 판정기를 키워도 우연 수준입니다

4.2.11 D3 손실 정답지 대비 | 60건, 보고→ORCHESTRATOR

측정 방식	순위 상관	실질 손실 AUROC
8B 사실 추출 + 집합 비교 (이전 D3)	0.61	0.62
원문 값 토큰 겹침	0.70	0.75
LLM 직접 비교, gpt-oss-20b (채택)	0.71	0.74

실질 손실 12건 중 8건은 누락이 아니라 바꿔 말하기였습니다. "기록을 남김"이 "처리 완료"로 바뀌는 식이라 값 집합의 차로는 보이지 않습니다. 지시→전제 경계는 정답 fidelity 0.97로 손실이 거의 없고, 실패 라벨 대비 AUROC는 0.53으로 실패 예측기로는 약합니다.

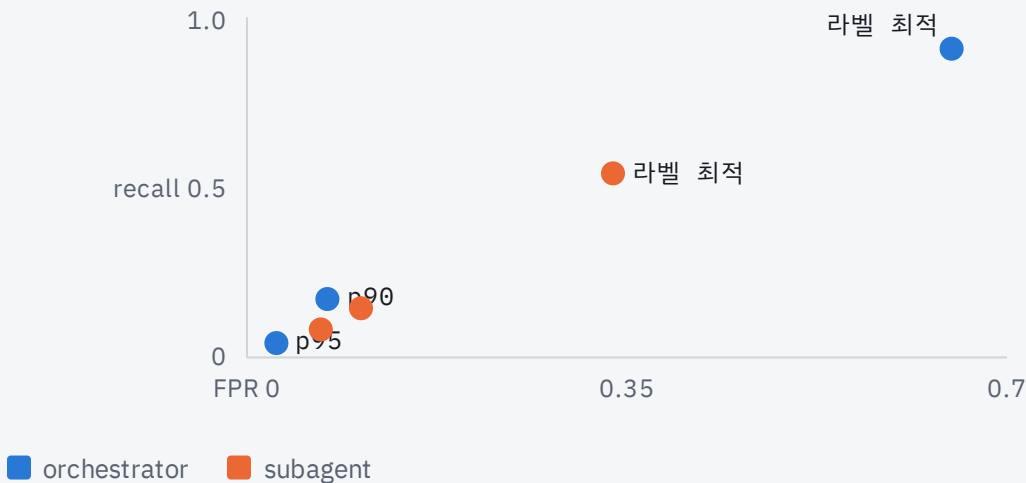
4.2.13 D4 판정기 세 종 비교 | AIRLINE SUBAGENT AUROC (95 % CI)



오탐의 정체는 "Next step: check LQ940Q" 같은 진행 문장과 "reservation is active" 같은 무해한 추정을 근거 없는 주장으로 센 것입니다. D4는 모든 주장의 근거를 묻고 라벨은 실패를 부른 오류만 세므로 두 질문이 다릅니다. 판정기 크기가 아니라 검사 정의의 문제이며, 결과에 채택하지 않았습니다.

D1은 순위 지표로 쓰고, 판정 주체는 탐지 대상마다 데이터로 정했습니다

4.2.6 임계값 실험 | 백분위 대 라벨 최적 (D1 1 - CONFIDENCE)



라벨 최적 임계값은 confidence 0.995 부근, 곧 조금이라도 흔들린 결정을 전부 잡는 지점이라 FPR이 0.34–0.65입니다. 상위 5–10 % 규칙은 recall을 0.04–0.17로 떨어뜨립니다. D1은 검토 우선순위로 쓰고 단독 알람은 두지 않습니다. D2는 0/1이라 라벨 없이 운영해도 잃는 것이 없습니다.

4.2.12 판정 방식 대조 | 같은 질문을 LLM에게 시키면

D1 · ORCHESTRATOR	전체 / DECISIVE		SUBAGENT	
코드 (실행 모델 logprob)	0.63 / 0.75		0.54	
LLM-only, gpt-oss-20b	0.64 / 0.77		0.62	
LLM-only, Qwen3-32B	0.60 / 0.68		0.65	

D2 · SUBAGENT	정밀도	재현율	F1	FPR
코드	0.62	0.48	0.54	0.07
LLM-only, gpt-oss-20b	0.44	0.46	0.45	0.12
LLM-only, Qwen3-32B	0.40	0.46	0.43	0.14

D1 orchestrator에서는 코드와 LLM이 같은 수준이라 확률 신호가 자기 판정 편향이 아님을 보여 줍니다. D2에서는 코드가 모든 지표에서 앞서고, D3에서는 반대로 LLM 직접 비교가 낮습니다. "판정은 코드가 한다"는 원칙은 구조화된 행동 지점에서만 성립합니다.

탐지기의 성능은 탐지기보다 어느 역할·어느 탐지 대상에 붙이는가에 달려 있습니다

	ORCHESTRATOR	SUBAGENT	판정
D1 결정 분포 (코드, 확률)	decisive 0.75 · 트레이스 max 0.81	0.54 · decisive 0.36 (우연)	orchestrator 결정에 배정
D2 행동 근거 (코드, 기록 대조)	플래그 2건 (recall 0.04) · 위임 조건은 정밀도 0.91	F1 0.54 · FPR 0.07	subagent 도구 지점과 orchestrator 위임에 배정
D3 handoff 정보 손실 (LLM)	보고→orchestrator 경계에서 손실 계속 성립 · 지시→전제는 손실 자체가 없음		경계 프로파일용, 실패 알람용 아님
D4 보고 근거 (LLM)	—	판정기 3종 모두 우연 수준	미채택, 검사 재정의 필요

4.3 해석 | 같은 질문, 역할마다 다른 실패의 모양

orchestrator의 실패는 결정의 흔들림으로, subagent의 실패는 확신에 찬 잘못된 호출로 나타납니다. 감사 파이프라인은 모든 스텝에 모든 탐지기를 돌리는 구조가 아니라 역할과 탐지 대상에 탐지기를 배정하는 라우팅 구조여야 합니다.

4.2.10 시스템 단위 집계 | 프로파일이 조치를 지목

db_agent는 도구 호출의 20 %가 근거 없는 인자였고 report→orchestrator 경계의 평균 fidelity는 0.44였습니다. 이런 프로파일이 프롬프트 수정이나 subagent 분리 같은 조치의 대상을 바로 지목합니다.

4.3 해석 | 측정 잡음이 뜻하는 것

같은 요청도 캐시 상태에 따라 confidence가 약 0.005 흔들립니다. 라벨 최적 임계값이 바로 그 잡음 수준에 있으므로 재현 가능한 경계가 아니며, D1의 유효 자릿수는 소수 둘째 자리까지입니다.

무엇을 주장하지 않으며, 다음에 무엇을 할 것인가

4.3 한계

- **한 도메인 30회.** 셀당 decisive 양성 8–16건, CI가 넓음. CI가 겹치는 곳에서 탐지기 간 순위를 주장하지 않음
- **라벨러가 사람이 아님.** decisive 일치 6/6이지만 이벤트 집합 Jaccard 0.56. 손실 정답지도 모델이 만듦
- **보고 지점은 아직 잡지 못함.** D4는 세 판정기 모두 우연 수준
- **단일 모델 계열과 로깅 계약.** 실행·시뮬레이터·채점이 모두 Qwen3. 출력만 남는 로그에서는 돌지 않음
- **순차 위임 그래프만.** 병렬 fan-out, 에이전트 간 직접 통신은 다루지 않았음

4.3 향후 방향

보고 지점

D4 검사 재정의

판정 대상을 결론 문장으로 좁히고 근거에 정책 문서를 포함해 다시 잼

보고 지점 · 코드

D7 재도출

환불 조건 같은 규칙을 코드로 옮겨 보고의 결론을 입력값에서 다시 계산. LLM 없이 추론 오류를 잡을 후보

행동 지점

D2 (1)의 정밀도

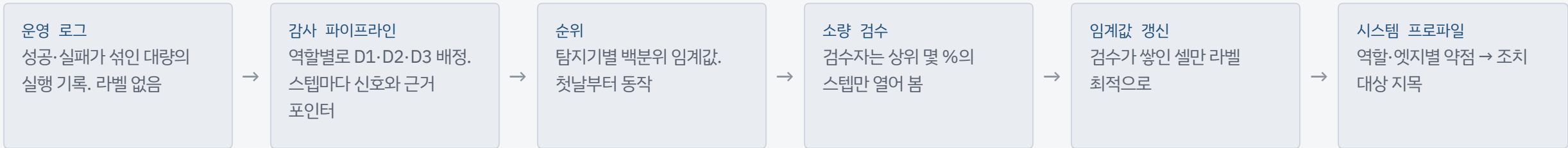
지시문 정규식을 구조화된 위임 스키마나 소형 모델 추출로 바꿔 오탐 14건을 줄임

일반화

도메인·그래프 확장

orchestrator 결정이 많은 검색·코딩 도메인, 병렬 fan-out에서 "역할이 가른다"는 결론이 유지되는지 확인

운영 로그를 사후 검토하는 제품에 그대로 있습니다



트레이스가 아니라 시스템을 평가

에이전트별 저신뢰 결정 비율과 D2 플래그 비율, handoff 앳지별 fidelity와 자주 빠지는 값이 나옵니다. "어디가 약한가"의 답이 트레이스 목록이 아니라 역할과 앳지 위의 프로파일이라 프롬프트 수정, 도구 스키마 변경, subagent 분리 같은 조치로 바로 번역됩니다.

로깅 계약이 곧 온보딩 체크리스트

스텝마다 모델이 본 입력 전체, 응답 본문, 도구 인자와 결과 원문, 공유 상태 스냅샷, 에이전트 식별자. 여기까지로 D2와 D3이 돌고, 실행 모델의 logprob 접근이 더해지면 D1이 켜집니다.

감사 가능

결정·행동 지점의 판정이 코드이므로 재현되고, 이의 제기에 근거 포인터로 답하며, 규칙 변경의 영향을 재실행으로 확인합니다. LLM 판정은 D3 하나뿐이며 누락·변형된 사실 목록으로 추적됩니다.

무엇을 잴고 무엇이 나왔는가

orchestrator가 subagent에게 일을 맡기는 시스템에서, 실행 기록만 보고 정답이나 라벨 없이 어느 스텝에서 어떤 실패가 일어났는지 짚어낼 수 있는지를 물었습니다. 기록을 결정, 행동, handoff 경계의 세 탐지 대상으로 나누고 탐지 대상마다 다른 검사를 두었습니다.

τ-bench airline 30회를 실행하고 탐지기 출력을 보지 않은 라벨러가 오류 215건을 달았습니다. 이 트레이스와 라벨은 네 실패 유형을 모두 담은 평가 데이터셋으로 함께 제출합니다.

orchestrator의 결정이 흔들린 정도는 실패를 부른 스텝을 AUROC 0.75로 가려내지만 subagent에서는 우연 수준입니다. subagent는 행동 근거 검사가 오류의 절반 가까이를 정밀도 0.62로 잡습니다. handoff 정보 손실은 계속으로 성립하지만 실패 예측력은 약하고, 보고 지점의 근거 판정은 판정기 세 종 모두 우연 수준이었습니다. 탐지기의 성능은 탐지기 자체보다 어느 역할과 탐지 대상에 붙이는가에 달려 있으며, 감사 파이프라인은 역할별로 검사를 배정하는 구조여야 합니다.

탐지기	역할	주 지표	판정
D1 결정 분포	orchestrator	AUROC 0.63 · decisive 0.75	작동
D1 결정 분포	subagent	0.54 · decisive 0.36	신호 없음
D2 행동 근거	subagent	F1 0.54 (정밀도 0.62 · 재현율 0.48 · FPR 0.07)	작동 (부분)
D2 행동 근거	orchestrator (위임)	정밀도 0.91 · 재현율 0.22	고정밀·저재현
D3 정보 손실, 보고 →orchestrator	orchestrator	순위 상관 0.71 · 실질 손실 AUROC 0.74	계측 성립
D3 정보 손실, 지시→전제	orchestrator	정답 fidelity 0.97	손실 없는 경계
D4 보고 근거	subagent	AUROC 0.47 / 0.55 / 0.50	미채택

AIME 2026은 왜 본문에서 뺐는가

같은 하네스, 30회

- orchestrator → solver, verifier (run_python), submit_answer로 한 번 제출
- 성공 7 / 실패 23 · 라벨 이벤트 219건 (tool 134 · handoff 16 · reasoning 69)
- solver가 풀이의 거의 전부를 수행. 라벨된 orchestrator 오류는 6건 (airline은 100건)
- subagent의 행동이 사실상 강제됨. run_python을 부르거나 보고하는 것 외에 선택지가 없음

orchestrator→subagent 분해가 형식적이어서 멀티에이전트 과제로 부적합하다고 판단했고, 역할 × 도메인 교훈의 대비 사례로만 씁니다.

결과가 AIRLINE과 반대 방향

지표	AIME 2026	AIRLINE
orchestrator D1, 트레이스 max AUROC	0.34	0.81
D2 subagent 정밀도 / 재현율 / F1	0.91 / 0.42 / 0.57	0.62 / 0.48 / 0.54
D2 subagent 귀속 에이전트 일치	0.86	0.25
repeat 로그 검사 정밀도	0.86	0.22

실패의 전형이 "solver가 run_python을 반복하거나 부르지 않고 답을 보고"라 도구 지점 검사만 강합니다.

결과를 바꾼 결정 여섯 가지

항목	결정	이유
A.4	내장 task 대신 이름 있는 래퍼 도구	task는 대상 subagent를 인자에 숨겨 D1이 "누구를 부르는가"를 썰 수 없음
A.6	D1 채점을 decode 경로 단일 방법으로	prefill 경로와의 차이가 bf16 반올림 잡음과 같은 크기. 어느 쪽도 기준이 될 수 없음
A.13-14	발언 값 대조·수식 검사 삭제, D2를 세 어긋남의 OR 단일 플래그로	0/1 검사를 어긋남별 AUROC로 매긴 평가가 잘못. 지어낸 값은 발언이 아니라 인자에 있었음
A.12	AIME 2026을 부록으로	orchestrator 결정이 6건뿐. 멀티에이전트 과제로 부적합
A.18-19	D3을 실패 예측기에서 정보 손실 계측기로, 판정은 LLM 직접 비교로	손실 정답지 60건 대비 LLM 비교가 8B 사실 추출보다 정확 (0.71 vs 0.61)
A.21	D4를 20B·32B 판정기로 재실행	8B 실패가 판정기 크기 탓인지 확인. 셋 다 우연 수준 → 검사 정의의 문제

라벨을 본 뒤에 고친 것은 두 가지뿐이며 모두 기록했습니다. D2 (1)의 매핑 오류 수정(A.14)과 orchestrator 위임 조건 추가(A.17). 둘 다 규칙은 라벨과 정답을 입력으로 받지 않습니다.

구현하지 않은 세 탐제기와 그 이유

D5

축약 손실

요약 미들웨어가 문맥을 줄일 때 사실이 빠지는지. 요약 직전 원문과 요약 결과를 D3 방식으로 비교. 이번 실험에서는 문맥이 16,000토큰에 이른 실행이 없어 요약이 발동하지 않았고 썬 표본이 없음.

D6

계획-행동 일치

orchestrator의 계획(todo, 지시문의 단계)과 실제 호출 순서 대조. 누락·추가·순서 뒤바뀜을 썬. todos가 subagent와 공유되지 않아 orchestrator 범위에서만 정의됨.

D7

재도출

환불 조건, 변경 수수료 같은 규칙을 코드로 옮겨 보고의 결론을 입력값에서 다시 계산. "근거 값에서 이 결론이 따라 나오는가"를 코드로 묻는 유일한 후보. 규칙 코드화 비용이 도메인마다 높.

세 탐지기 모두 완전 관측 기록 위에서 코드로 판정하도록 설계했습니다. D7이 보고 지점 공백을 메울 첫 후보입니다.

커밋된 산출물에서 모든 표를 다시 만들 수 있습니다

```
bash scripts/serve_qwen32b.sh & bash scripts/serve_qwen32b_score.sh & bash scripts/serve_gptoss20b.sh & # 실행 · D1 채점 · D3/D4 판정
python -m src.run --batch 1 --parallel 4 --resume # 실행 (이어서 --batch 2)
python -m src.audit.d1_decision --all --method stepwise; python -m src.audit.d2_action; python -m src.audit.d3_handoff --stats
python -m src.eval.labels --validate && python -m src.eval.metrics && python -m src.eval.thresholds && python -m src.eval.recovery
```

E 실행·채점 설정 | 모델 설정

- Qwen3-32B: 실행·시뮬레이터(thinking 끄, logprobs로 토큰 id 수신), D1 채점 (같은 가중치, 문맥 24,576토큰), LLM 비교군·D4 판정(thinking 켜)
- gpt-oss-20b: D3·D4 판정, 기본 reasoning. 실행 모델과 다른 계열
- Qwen3-8B: 비교군 전용. 모든 호출 temperature 0

B 재현 | 커밋 정책

- 원본 trace(steps.jsonl, tool_calls.sqlite, fs/)는 용량 때문에 미커밋. 스크립트로 재생성
- runs/index.csv, 실행 요약, audit/, labels/, eval/, 스크립트 전체는 커밋
- vLLM 0.9.2(실행·D1 채점), 0.11.0(gpt-oss-20b 판정)

무엇을 AI에게 맡기고 무엇을 사람이 결정했는가

단계	AI (CLAUDE CODE)	사람
설계	인터뷰로 스펙과 계획 v2.4 작성, 두 검토 에이전트의 합의본	연구 질문, 네 탐지 대상의 틀, 평가 조건, 계획 승인
구현·실행	하네스, 캡처, D1–D4, 평가·게이트 스크립트, 60회 실행과 채점	게이트 확인, 탐지기 정의의 채택·기각, 자원 배정
라벨링	서브에이전트 6개 블라인드 라벨, 독립 스팟체크, 손실 정답지	라벨 규약 승인, 반복 규약 확정
해석·문서	지표, 표, 원인 진단, 초안, 수치 대조, 출처 확인	AIME 부록화, D2 정의, D4 재실행, 이름과 구조, 문제 정의와 요약 직접 수정

기록

커밋 151건, 시간순 작업 로그, 계획 이탈 결정 21건(부록 A), 문서 게이트 `check_docs.py`. 본문의 수치는 `eval/metrics.md`와 `audit/` 출력에 다시 대조했습니다.

이 방식의 한계

라벨과 손실 정답지를 모델이 만들었으므로 사람 기준의 타당성은 아직 보이지 않았습니다. 에이전트가 쓴 초안에는 이전 결과가 남는 경우가 있어 수치 대조 절차가 필요했습니다. 결과 해석과 무엇을 결과로 인정할지의 판단은 사람이 맡는 편이 맞았습니다.